

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №4
По дисциплине "Веб-программирование"
Вариант 75

Выполнил студент группы Р3209
Евграфов Артём Андреевич
Проверил:
Барсуков Максим Андреевич

Санкт-Петербург 2025

Содержание

1. Задание варианта 75	2
2. Ход работы	2
3. Дополнительное задание	3
4. Вывод	3

1. Задание варианта 75

Лабораторная работа #3

Вариант 36734

Внимание! У разных вариантов разный текст задания!

Разработать приложение на базе JavaServer Faces Framework, которое осуществляет проверку попадания точки в заданную область на координатной плоскости.

Приложение должно включать в себя 2 facadelets-шаблона - стартовую страницу и основную страницу приложения, а также набор управляемых бинов (managed beans), реализующих логику на стороне сервера.

Стартовая страница должна содержать следующие элементы:

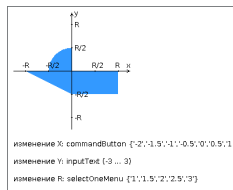
- "Шапку", содержащую ФИО студента, номер группы и номер варианта.
- Интерактивные часы, показывающие текущие дату и время, обновляющиеся раз в 9 секунд.
- Ссылку, позволяющую перейти на основную страницу приложения.

Основная страница приложения должна содержать следующие элементы:

- Набор компонентов для задания координат точек и радиуса области в соответствии с вариантом задания. Может потребоваться использование дополнительных библиотек компонентов – **ICEFaces** (префикс "ice") и **PrimeFaces** (префикс "p"). Если компонент допускает ввод заведомо некорректных данных (таких, например, как буквы в координатах точек или отрицательный радиус), то приложении должно осуществлять их валидацию.
- Динамически обновляемую картинку, изображающую область на координатной плоскости в соответствии с номером варианта и точки, координаты которых были заданы пользователем. Клик по картинке должен инициировать сценарий, осуществляющий определение координат новой точки и отправку их на сервер для проверки ее попадания в область. Цвет точек должен зависеть от факта попадания / непопадания в область. Смена радиуса также должна инициировать перерисовку картинки.
- Таблицу со списком результатов предыдущих проверок.
- Ссылку, позволяющую вернуться на стартовую страницу.

Дополнительные требования к приложению:

- Все результаты проверки должны сохраняться в базе данных под управлением СУБД HSQLDB.
- Для доступа к БД необходимо использовать ORM Hibernate.
- Для управления списком результатов должен использоваться Session-scoped Managed Bean.
- Конфигурация управляемых бинов должна быть задана с помощью параметров в конфигурационном файле.
- Правила навигации между страницами приложения должны быть заданы в отдельном конфигурационном файле.

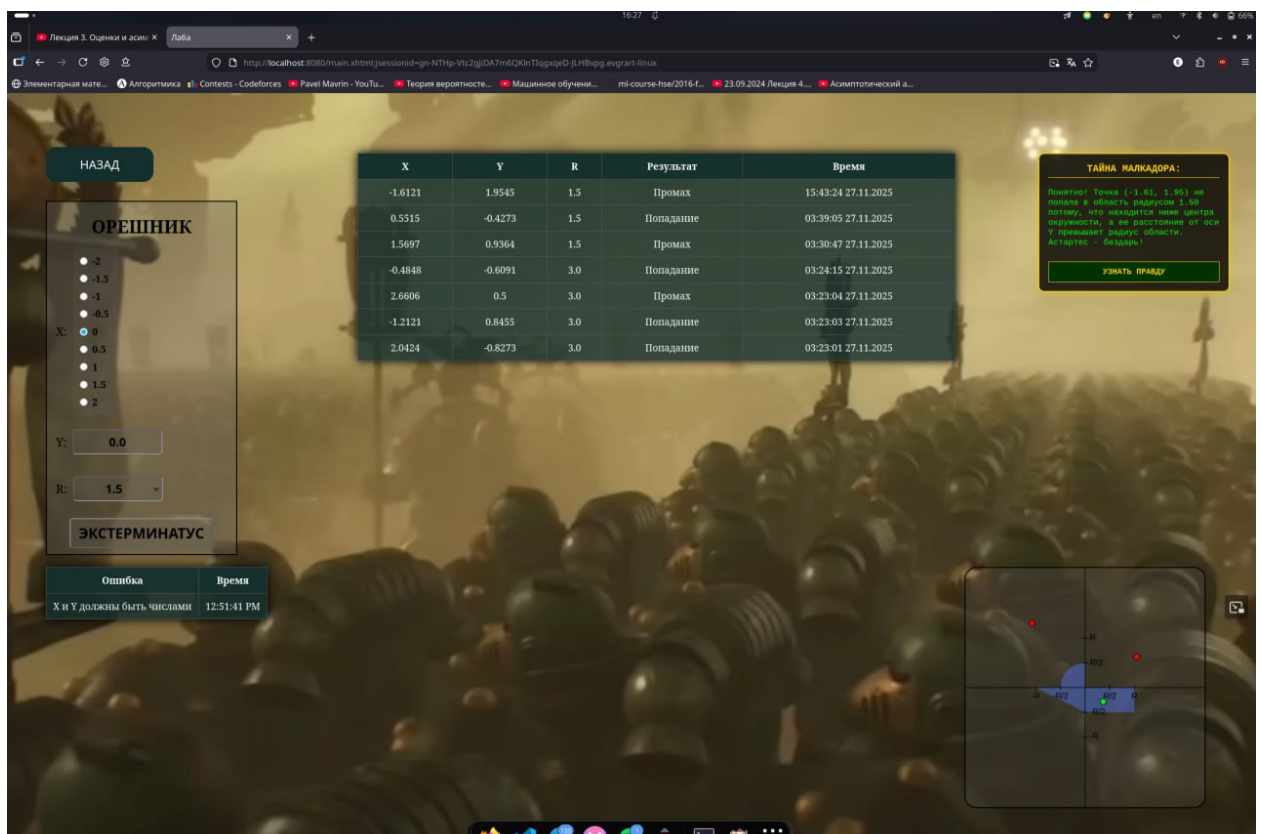


Вопросы к защите лабораторной работы:

1. Технология JavaServer Faces. Особенности, отличия от сервлетов и JSP, преимущества и недостатки. Структура JSF-приложения.
2. Использование JSP-страниц и Facelets-шаблонов в JSF-приложениях.
3. JSF-компоненты - особенности реализации, иерархия классов. Дополнительные библиотеки компонентов. Модель обработки событий в JSF-приложениях.
4. Конвертеры и валидаторы данных.

2. Ход работы

[Ссылка на код](#)



3. Дополнительное задание

Реализуйте расширенную логику проверки попадания точки в заданную область, при которой: Проверка выполняется аналитически, а не численно.

Каждый результат сопровождается машинно-читаемым доказательством, построенным как последовательность логически обоснованных шагов.

Корректность результата независимо подтверждается с помощью внешнего SMT-решателя (например, Z3).

Доказательство визуализируется на фронтенде в интерактивном виде.

Требования к бэку: 1. Символьное ядро: Используйте библиотеку Symja (или аналогичную) для символьных вычислений. На основе выбранной фигуры (предусмотренной вариантом задания) и переданных координат точки постройте цепочку преобразований, например: $x^2 + y^2 \leq r^2 \rightarrow 1^2 + 0.5^2 \leq 2 \cdot 2 \rightarrow 1.25 \leq 4 \rightarrow \text{true}$. Каждый шаг доказательства должен быть представлен как структурированный объект с полями: type (SUBSTITUTION, EVALUATION, INEQUALITY_CHECK), expression, result.

2. Формальная верификация через SMT: Сгенерируйте запрос в формате SMT-LIB v2, эквивалентный условию попадания. Выполните его с помощью Z3, запущенного в изолированном контейнере. Результат ответа (true/false) должен возвращаться только если он совпадает с выводом Z3. В противном случае - возвращайте HTTP 500 с диагностикой. Ограничьте время выполнения символьных вычислений и вызова Z3 до 500 мс. При превышении выбрасывайте TimeoutException, логируйте событие и возвращайте ошибку клиенту.

3. REST API: Эндпоинт /api/check должен возвращать JSON следующей структуры:

```
inside: true,  
proof: [  
  type: SUBSTITUTION, expression:  $x^2 + y^2 \leq r^2$ , substitution: x: 1, y: 0.5, r: 2 ,  
],  
smtVerified: true
```

Требования к фронтенду:

1. Интерактивный просмотр доказательства: отображайте цепочку шагов в раскрывающемся списке. При наведении или клике на шаг подсвечивайте соответствующую зону на холсте (например, область круга для шага с неравенством).

2. Используйте HTML Canvas (не статичное изображение) для отрисовки фигуры и точек.

3. Цвет точки должен отражать не только факт попадания, но и статус верификации (например, зелёный - подтверждено Z3, оранжевый - только локальная проверка, красный - расхождение).

Если `smtVerified == false`, отобразите предупреждение

4. Вывод

В ходе этой лабораторной работы я изучил современные подходы к веб программированию в джакартаЕЕ и в веб фреймворках.